

Herald



CodeGraph Integration

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	Initial CodeGraph integration per user mandate 2026-05-20 + Universal §11.4.78 (CodeGraph code-intelligence mandate). Herald is indexed (48 Go files / 775 nodes / 951 edges). Setup + validate scripts shipped. CLI-agent wiring documented for Claude Code (primary), OpenCode, Kimi CLI, Crush, Qwen Code. Anti-bluff <code>scripts/codegraph_validate.sh</code> proves the index returns real Herald symbols with 7/7 PASS at landing time.
Issues	none
Issues summary	—
Fixed	initial integration
Fixed summary	—
Continuation	Re-run <code>scripts/codegraph_setup.sh</code> after any non-trivial code change to keep the graph fresh. The setup script auto-detects existing <code>.codegraph/</code> and runs <code>sync</code> rather than full re-index.

Table of contents

- [§1. What CodeGraph is](#)
- [§2. Setup](#)
- [§3. Anti-bluff validation](#)
- [§4. CLI agent wiring](#)
 - [4.1 Claude Code \(primary\)](#)
 - [4.2 OpenCode](#)
 - [4.3 Codex CLI](#)
 - [4.4 Cursor](#)
 - [4.5 Kimi CLI](#)
 - [4.6 Crush](#)
 - [4.7 Qwen Code](#)

- [§5. Daily use](#)
- [§6. Forensic anchors + composition](#)

§1. What CodeGraph is

CodeGraph (@colbymchenry/codegraph, MIT, <https://github.com/colbymchenry/codegraph>) is a local pre-indexed knowledge graph for source code. Per its README benchmark, it produces $\approx 92\%$ fewer Claude Code tool calls and $\approx 71\%$ faster Explore-agent runs against 6 reference codebases.

For Herald specifically: instead of having an Explore subagent `grep + glob + Read 40 files` to answer "how does the constitution-rule Evaluator dispatch work?", CodeGraph returns the exact symbol locations and edges in one query.

Per Universal Constitution §11.4.78 (CodeGraph code-intelligence mandate) every consuming project should adopt CodeGraph. This document is Herald's adoption record.

§2. Setup

Prerequisites: Node.js 18+ on PATH.

```
cd <herald-root>
scripts/codegraph_setup.sh
```

The script is idempotent:

- If `.codegraph/` does not exist $\rightarrow$ runs `init` + full index.
- If `.codegraph/` exists $\rightarrow$ runs `sync` (incremental update of changed files).
- Always finishes with `status` to print the graph stats.

At landing time the index reports:

```
Nodes:      775
Edges:      951
DB Size:    1.54 MB
Files by Language: go (48)
```

The `.codegraph/codegraph.db` is gitignored (`.codegraph/.gitignore` ships its own ignore rules). `.codegraph/config.json` IS tracked — it carries the include/exclude globs.

§3. Anti-bluff validation

Per Universal §11.4 + §11.4.69, a CodeGraph install that "looks installed" but returns empty results for real Herald symbols is the exact bluff §11.4 forbids. The validation script asserts the index is functioning:

```
scripts/codegraph_validate.sh
```

Probes 6 known Herald symbols + 1 negative control. PASSES only when:

- Every positive probe (Evaluator, ConstitutionStore, Record, WithTenantContext, QuickstartBoot, Server) returns ≥ 1 hit AND

- The negative control (a definitely-absent symbol) returns ≤ 1 hit.

At landing time: 7/7 **PASS**.

Re-run after any non-trivial code change. A regression here means the index is stale — re-run `scripts/codegraph_setup.sh` to refresh.

§4. CLI agent wiring

CodeGraph exposes itself to AI agents via the **Model Context Protocol** (MCP) server. The agent connects to `codegraph serve --mcp` and gains a `codegraph` tool that maps to the same query semantics as the CLI.

4.1 Claude Code (primary)

Add to `.claude/settings.json` (project-local) OR `~/.claude/settings.json` (user-global):

```
{
  "mcpServers": {
    "codegraph": {
      "command": "npx",
      "args": ["-y", "@colbymchenry/codegraph", "serve", "--mcp"],
      "env": {}
    }
  }
}
```

Restart Claude Code. Verify with `/mcp` — `codegraph` should appear as a connected server. Explore agents will now have access to the `codegraph` tool.

4.2 OpenCode

OpenCode honors the same MCP configuration. Add to `~/.opencode/config.json`:

```
{
  "mcpServers": {
    "codegraph": {
      "command": "npx",
      "args": ["-y", "@colbymchenry/codegraph", "serve", "--mcp"]
    }
  }
}
```

4.3 Codex CLI

Per the CodeGraph README, Codex CLI is supported out-of-the-box by the package's interactive installer:

```
npx -y @colbymchenry/codegraph install
```

Select Codex CLI when prompted.

4.4 Cursor

Cursor supports MCP via its settings UI: **Settings** → **MCP** → **Add Server**, then paste the same command + args as in §4.1.

4.5 Kimi CLI

Kimi CLI's MCP support is configured via `~/.kimi/mcp.json` (subject to upstream changes):

```
{
  "servers": {
    "codegraph": {
      "command": "npx",
      "args": ["-y", "@colbymchenry/codegraph", "serve", "--mcp"]
    }
  }
}
```

If Kimi CLI's MCP path differs from the canonical Claude Code shape (the spec is still evolving), the operator **MUST** adjust per the active Kimi CLI version's docs. Filed as HRD-083 if the wiring needs Herald-side code.

4.6 Crush

Crush's MCP wiring follows the same pattern. Consult Crush's MCP docs for the exact config-file path; the command/args payload above is the same.

4.7 Qwen Code

Qwen Code MCP support is documented in the parent constitution submodule's `QWEN.md`. Add the same MCP server block to `~/.qwen/mcp.json` (or per active Qwen Code version's MCP config path):

```
{
  "mcpServers": {
    "codegraph": {
      "command": "npx",
      "args": ["-y", "@colbymchenry/codegraph", "serve", "--mcp"]
    }
  }
}
```

§5. Daily use

- After significant changes: `scripts/codegraph_setup.sh` (runs sync).
- Quick stats: `scripts/codegraph_setup.sh status`.
- Anti-bluff re-validation: `scripts/codegraph_validate.sh`.

- Visualise the graph: `npx -y @colbymchenry/codegraph visualize` (opens browser).

If an Explore subagent reports stale results despite CodeGraph being present, run `sync` first. A `mark-dirty + sync-if-dirty` post-commit hook can automate this — track under a follow-up HRD if Herald needs it (HRD-084).

§6. Forensic anchors + composition

- **Universal §11.4.78** — CodeGraph code-intelligence mandate (constitution submodule `Constitution.md`).
- **§11.4.74** — Submodule-catalogue-first discovery. CodeGraph is a third-party MIT-licensed Submodule-equivalent (npm package); the Catalogue-Check verdict for "code-intelligence pre-indexing" is `extend external standard` rather than `extend vasic-digital` because no `vasic-digital/*` repo provides this capability.
- **§11.4 / §11.4.69** — Anti-bluff covenant. `scripts/codegraph_validate.sh` is the captured-evidence proof per §11.4.5 — every PASS here means a real index returning real Herald symbol locations.
- **§11.4.65** — Universal Markdown export. This guide's HTML + PDF + DOCX siblings ship in the same commit.
- **§11.4.20** — Subagent-driven-by-default. CodeGraph is the substrate that makes the subagent default cheap ($\approx 92\%$ fewer tool calls per its benchmarks); the two clauses compose: subagents-by-default for parallelism + CodeGraph for per-subagent efficiency.

Sources verified

This guide documents the external **CodeGraph** tool (@colbymchenry/codegraph, MIT) plus Herald's own setup/validate scripts. Per §11.4.99 the external-tool claims were cross-referenced against the tool's published source before this footer landed.

Verified 2026-05-30: <https://github.com/colbymchenry/codegraph> (CodeGraph upstream — package name, MIT license, `serve --mcp / install / sync / status / visualize` subcommands, the $\approx 92\%$ -fewer-tool-calls / $\approx 71\%$ -faster-Explore benchmark claims, and the per-agent MCP-config shapes cited in §4); <https://www.npmjs.com/package/@colbymchenry/codegraph> (npm distribution invoked via `npx -y @colbymchenry/codegraph`).

In-repo (read 2026-05-30): `scripts/codegraph_setup.sh`, `scripts/codegraph_validate.sh` (the Herald-side wrappers), and the `.codegraph/` index layout (`codegraph.db + config.json`, both gitignored). The MCP-client config paths for OpenCode / Cursor / Kimi CLI / Crush / Qwen Code are upstream-evolving and **MUST** be reconfirmed against each client's active-version docs before use — flagged inline in §4.5/§4.6/§4.7.

Re-verification cadence (per §11.4.99 (C)): CodeGraph is a package-manager-distributed third-party tool (risk-classified) → **90-day staleness** (next due **2026-08-28**); re-verify the npm package name + subcommand surface + MCP config shapes on any CodeGraph major-version bump.